1 · Subdomínios

Classificação a partir do enunciado (oficina de médio porte; foco em ordens de serviço, clientes e peças). Cada subdomínio corresponde a uma pasta de mesmo nome em `Oficina.Domain`, `Oficina.Application` e `Oficina.Infrastructure`.

Board no Miro: https://miro.com/app/board/uXjVEVRxgE4=/. O conteúdo é o mesmo deste PDF; a fonte é `docs/ddd/fontes/`.

PRINCIPAL

Ordens de Serviço e Orçamento

`ServiceOrders`

- Abertura da O.S. por CPF/CNPJ e placa
- Diagnóstico com inclusão de serviços, peças e insumos
- Orçamento automático, envio e aprovação/recusa (balcão ou link)
- Máquina de 6 status com histórico
- Consulta pública do andamento e tempo médio de execução

Por quê: é o que resolve as dores do enunciado (priorização, status, orçamentos e autorizações).

SUPORTE

Cadastro de clientes e veículos

`Customers`

- Cliente com CPF/CNPJ validado
- Veículo com placa antiga/Mercosul
- Histórico preservado (soft delete)

SUPORTE

Catálogo de serviços

`Catalog`

- Serviço com preço base e tempo estimado
- Desativar / reativar

SUPORTE

Peças, insumos e estoque

`Inventory`

- Peça (`Part`) e insumo (`Supply`)
- Entrada e baixa com movimentações
- Estoque mínimo

GENÉRICO

Identidade e acesso

`Identity` / `Users`

- Login JWT, BCrypt
- Perfis Admin, Attendant, Mechanic

GENÉRICO

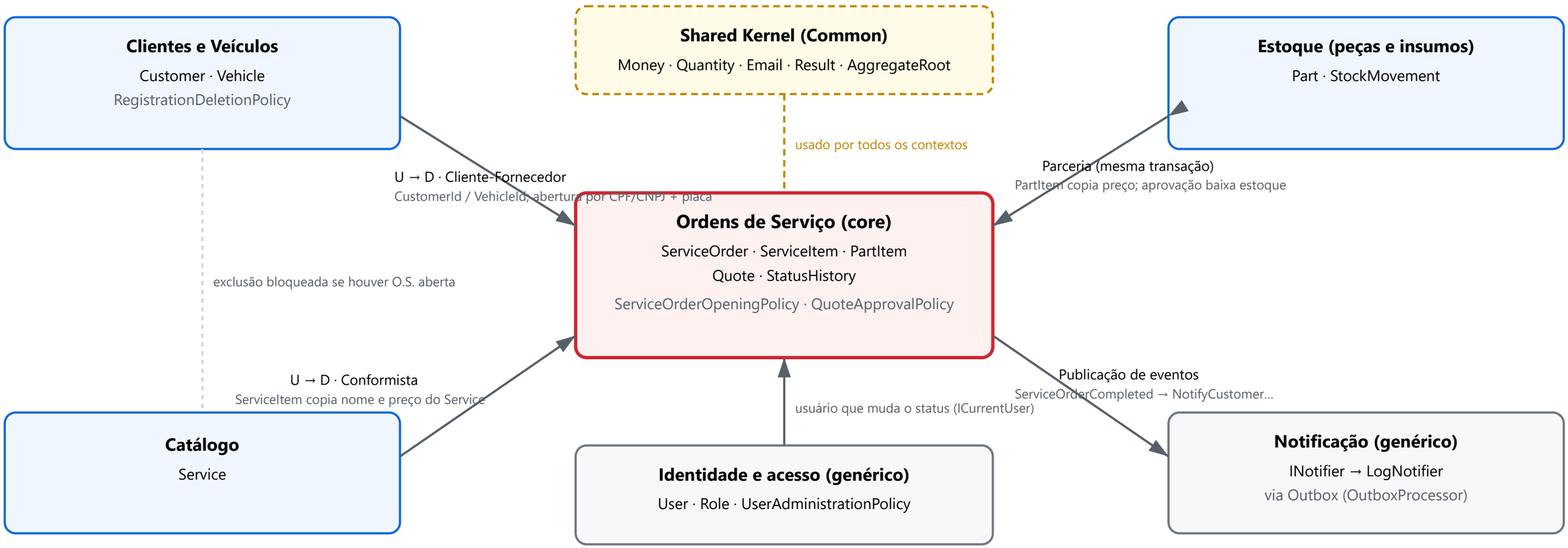
Notificação

`INotifier` / `LogNotifier`

- Aviso de O.S. finalizada (hoje em log)

2 · Mapa de contextos

Monólito em camadas: os bounded contexts são módulos (namespaces) do mesmo processo e do mesmo banco. Eles se referenciam por id e se integram por chamadas diretas no domínio ou por eventos de domínio gravados no Outbox.



Relação	Padrão	Como integra no código
Clientes e Veículos → Ordens de Serviço	Cliente-Fornecedor	`OpenServiceOrderCommandHandler` busca `Customer` por `Document` e `Vehicle` por `LicensePlate`; a O.S. guarda `CustomerId`/`VehicleId`. `RegistrationDeletionPolicy` consulta `IServiceOrderRepository` no sentido inverso.
Catálogo → Ordens de Serviço	Conformista	`ServiceOrder.AddServiceItem(Service, …)` copia nome e `BasePrice`; mudança posterior de preço não altera a O.S.
Estoque ↔ Ordens de Serviço	Parceria	`ServiceOrder.AddPartItem(Part, …)` exige `Part.HasAvailable`; `QuoteApprovalPolicy` chama `Part.DeductForServiceOrder` no mesmo commit.
Ordens de Serviço → Notificação	Publicação de eventos (Outbox)	`OutboxWriter` grava os eventos na transação; `OutboxProcessor` entrega `ServiceOrderCompleted` a `NotifyCustomerOnServiceOrderCompleted` → `INotifier`.
Todos → Common	Shared Kernel	`Money`, `Quantity`, `Email`, `Result/Error`, `Entity/AggregateRoot`, `IDomainEvent`.

3 · Modelo de domínio (agregados, entidades e objetos de valor)

Raiz do agregado em negrito. Agregados se referenciam só por id. Todas as entidades herdam `Entity` (auditoria `createdAt/updatedAt` e soft delete `deletedAt`).

Agregado **ServiceOrder** (`ServiceOrders`)

Tipo	Nome	Dados / regras
Raiz	**ServiceOrder**	`Number`, `CustomerId`, `VehicleId`, `Status`, `ProblemDescription` (≥ 3), `TotalAmount`, `ExecutionStartedAt`, `CompletedAt`, `DeliveredAt`, `rowversion`
Entidade	ServiceItem	`ServiceId`, descrição, `UnitPrice`, quantidade inteira > 0
Entidade	PartItem	`PartId`, descrição, `UnitPrice`, `Quantity`
Entidade	Quote	`Version`, `QuoteStatus { Pending, Approved, Rejected }`, totais de serviços/peças, `ApprovalToken`, motivo da recusa
Entidade	StatusHistory	status anterior → novo, usuário, nota, data
Objeto de valor	ServiceOrderNumber	sequencial > 0 (sequence do SQL Server)
Objeto de valor	ApprovalToken	hash SHA-256 (64 hex) do token enviado ao cliente
Enum	ServiceOrderStatus	`Received → InDiagnosis → WaitingApproval → InProgress → Completed → Delivered`
Serviço de domínio	ServiceOrderOpeningPolicy	veículo do cliente; 1 O.S. aberta por veículo
Serviço de domínio	QuoteApprovalPolicy	aprova e baixa o estoque das peças
Eventos	`ServiceOrderOpened`, `QuoteSent`, `QuoteApproved` (com `ApprovedPart`), `QuoteRejected`, `ServiceOrderCompleted`	

Agregado **Customer** (`Customers`)

- Nome, `Document` (CPF/CNPJ, imutável), `DocumentType`, `Email?`, `Phone`, `Address?`

Agregado **Vehicle** (`Customers`)

- `CustomerId`, `LicensePlate`, marca, modelo, `VehicleYear`

Objetos de valor: `Document`, `Phone` (10–11 dígitos), `Address` (CEP 8 dígitos, UF), `LicensePlate`, `VehicleYear` (≥ 1900, ≤ ano seguinte).
Serviço: `RegistrationDeletionPolicy`.

Agregado **Service** (`Catalog`)

- Nome (2–120), descrição, `BasePrice` (> 0), `EstimatedMinutes` (> 0), `IsActive`

Agregado **Part** (`Inventory`)

- `PartCode`, nome, `PartType { Part, Supply }`, unidade, `UnitPrice`, `StockQuantity`, `MinimumStock`, `IsActive`, `rowversion`
- Entidade `StockMovement`: `StockMovementType { Entry, Deduction }`, quantidade, saldo após, motivo, `ServiceOrderId?`

Agregado **User** (`Identity`)

- Nome, `Email` único, `PasswordHash` (BCrypt), `Role { Admin, Attendant, Mechanic }`, `IsActive`

Serviço: `UserAdministrationPolicy` (não gerencia a própria conta; sempre resta um Admin ativo).

Shared Kernel (`Common`)

- `Money`: ≥ 0, 2 casas, arredondamento meio para cima
- `Quantity`: > 0, até 3 casas
- `Email`, `Result`, `Error`, `IDomainEvent`

4 · Domain Storytelling · As-is (como a oficina trabalha hoje)

Cenas do processo manual descrito no enunciado (anotações e planilhas). Em vermelho, a dor que o sistema resolve.

As-is 1 · Atendimento e cadastro

Cliente ❶ *relata defeito e entrega* Veículo ❷ *ao* **Atendente** ❸ *anota dados e defeito na* Ficha em papel / planilha — Histórico de clientes e veículos se perde; não há busca por CPF/CNPJ nem conferência da placa

As-is 2 · Diagnóstico e orçamento

Mecânico ❶ *examina o veículo e anota* Serviços e peças necessários ❷ *passa ao* **Atendente** ❸ *soma à mão e liga para o* **Cliente** — Orçamento manual, sujeito a erro; aprovação verbal sem registro

As-is 3 · Execução e controle de peças

Mecânico ❶ *retira da prateleira* Peças e insumos ❷ *às vezes lança na* Planilha de estoque — Saldo da planilha não bate com a prateleira; peça falta no meio do serviço

As-is 4 · Acompanhamento e entrega

Cliente ❶ *liga perguntando o andamento ao* **Atendente** ❷ *pergunta ao* **Mecânico** ❸ *avisa que está pronto e entrega o* Veículo — Ninguém sabe o status sem perguntar; não há medida do tempo de execução; priorização errada

5 · Domain Storytelling · To-be (com o sistema)

Mesmas cenas usando a API. Em verde, o que o sistema faz.

To-be 1 · Abertura da O.S.

Atendente ❶ *cadastra* Cliente (CPF/CNPJ) ❷ *e* Veículo (placa, marca, modelo, ano) ❸ *abre a* O.S. com o relato ❹ *no* Sistema: valida documento e placa, impede 2ª O.S. aberta no veículo, status Recebida

To-be 2 · Diagnóstico e orçamento

Mecânico ❶ *inicia o diagnóstico e inclui* Serviços, peças e insumos ❷ **Atendente** ❸ *envia o* Orçamento — Sistema: confere saldo, soma o total, gera o link (token), status Aguardando aprovação

To-be 3 · Decisão do cliente e execução

Cliente ❶ *aprova ou recusa pelo link (ou no balcão)* Orçamento — Sistema: aprovado → Em execução e baixa do estoque na mesma transação; recusado → volta a Em diagnóstico — **Mecânico** ❷ *finaliza a* O.S. — Finalizada; aviso ao cliente (log)

To-be 4 · Acompanhamento, entrega e gestão

Cliente ❶ *consulta com número + CPF/CNPJ* Status, itens e orçamento — **Atendente** ❷ *entrega o* Veículo — Entregue — **Admin** ❸ *consulta* Tempo médio de execução, O.S. por status, peças abaixo do mínimo

6 · Arquitetura (C4)

Back-end monolítico em camadas, empacotado em um container Docker com SQL Server ao lado (`docker-compose.yml`).

Nível 1 · Contexto

Equipe da oficina
Admin, Atendente, Mecânico · JWT

Cliente
sem login: consulta por número + CPF/CNPJ e decide pelo link

→

Oficina API (mecanicaOS)
Gestão de O.S., clientes, veículos, serviços, peças e estoque

→

Notificação ao cliente
hoje `LogNotifier` (log); e-mail/WhatsApp é próximo passo

Nível 2 · Containers

Oficina.Api
ASP.NET Core (.NET 10) · REST · Swagger · JWT · imagem do `Dockerfile`, porta 8080. Inclui o `OutboxProcessor` (BackgroundService).

↔

SQL Server 2022
EF Core Migrations · transações ACID · `rowversion` · índices únicos filtrados · tabela de Outbox

Nível 3 · Componentes da API (camadas)

Oficina.Api
Controllers `Admin/*`, `Public/*`, `Auth`; policies de perfil; ProblemDetails; rate limit do login

Oficina.Application
Commands e queries do MediatR, validators (FluentValidation), behaviors de log e validação, DTOs e mappers (Mapperly), handler de evento

Oficina.Domain
Agregados, objetos de valor, serviços de domínio, eventos, catálogos de erro, interfaces dos repositórios. Não depende de nenhuma camada.

Oficina.Infrastructure
DbContext e configurações, repositórios, consultas, `UnitOfWork`, Outbox, JWT, BCrypt, `LogNotifier`, seeders

Dependências: Api → Application → Domain; Infrastructure → Application/Domain (implementa as interfaces). A regra é verificada por `tests/Oficina.UnitTests/Architecture/LayerDependencyTests.cs`.